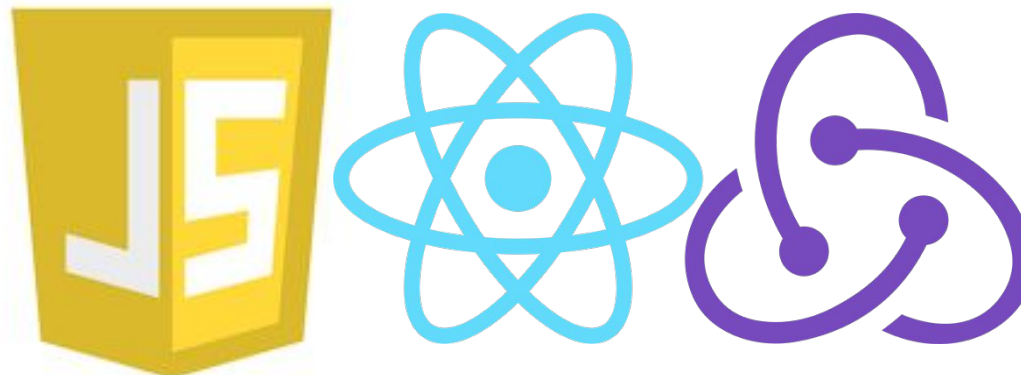


Лекція 14

Сучасний фронтенд

JavaScript. React (with hooks) + Redux



1. React. Hooks.

Hooks (хуки) — нововведення у React 16.8, що дозволяє використовувати стан та інші можливості React без написання класів.

// Компонент - клас

```
class Example extends React.Component {  
  render() {  
    return (  
      <div>  
        { this.props.field1 }  
      </div>  
    );  
  }  
}
```

// Компонент - функція

```
const Example2 = ({  
  field1,  
}) => (  
  <div>  
    { field1 }  
  </div>  
);
```

2. React. Hooks. Правила.

Хуки - звичайні JavaScript-функції, але існує 3 правила, яким потрібно слідувати

1. Використовуйте хуки лише на верхньому рівні.

Не викликайте хуки всередині циклів, умовних операторів чи вкладених функцій. Натомість завжди використовуйте хуки тільки всередині React-функцій, до повернення будь-якого значення з них.

1. Викликайте хуки лише з React-функцій.

Не викликайте хуки із звичайних функцій JavaScript. Натомість можна:

- Викликати хуки із функціонального компоненту React.
- Викликати хуки з хука користувача

1. При створенні хука користувача, його ім'я повинно починатися з **use...**

3. React. Hooks. useState.

Раніше, якщо ви писали функціональний компонент і усвідомлювали, що потрібно наділити його станом, вам доводилося перетворювати цей компонент на клас. Тепер же можна використовувати хук **useState** всередині існуючого функціонального компонента:

```
import React, { useState } from 'react';
const Example = ({
  field1,
}) => {
  const initialState = { name: 'Petr' };
  const [state, setState] = useState(initialState);
  return (
    <button
      onClick={() => setState(prevState => ({
        ...prevState,
        name: 'Not Petr'
      })))}
    >
      Change name
    </button>
  );
};
```

4. React. Hooks. useEffect.

Хук **useEffect** дає можливість виконувати побічні ефекти у функціональній компоненті.

```
import React, { useEffect, useState } from 'react';
import fetchData from '../actions/data'

const Example = ({ fetchedData, fromDate }) => {
  const [state, setState] = useState({ data: null });

  useEffect(() => {
    fetchData(fromDate);
  }, [fromDate]);

  useEffect(() => {
    setState(prevState => ({
      ...prevState,
      data: {
        ...fetchedData, // Можуть бути будь-які конвертації формату даних
      },
    }));
  }, [fetchedData]);

  return ( <div/> );
};
```

5. React. Hooks. useEffect.

Хук **useEffect** замінює відразу 3 методи життєвого циклу з попередньої версії реакту:

- componentDidMount
- componentDidUpdate
- componentWillUnmount

Хук **useEffect** - це функція, яка приймає 2 аргументи:

- Перший аргумент - це callback, який має бути виконаний після кожного рендеру.
- Другий аргумент - це масив змінних, за якими слідкуватиме **useEffect**. Функція callback (перший аргумент) буде виконана тільки в тому випадку, якщо зміниться значення хоч однієї зі змінних, переданих до цього масиву.

Таким чином, якщо ми хочемо, щоб callback з **useEffect**'а спрацьовував після кожного рендеру, то ми не повинні передавати другий аргумент.

Якщо ми хочемо щоб callback викликався лише після першого рендера, ми повинні передати другим аргументом порожній масив.

6. React. Hooks. Детальніше.

Якщо ви хочете отримати більш розгорнуту інформацію про такі базові хуки як **useState**, **useEffect**, та деякі інші хуки, про які не буде згадано в даній презентації, то це можна зробити за посиланням:

<https://17.reactjs.org/docs/hooks-intro.html>

7. React. Hooks. Користувальницькі хуки.

Т.к. хук - це звичайна функція, ми можемо створювати і свої хуки.

Давайте створимо власний хук, який рахуватиме факторіал числа:

```
import React from 'react';
```

```
const calculate = (number) => {
```

```
  if (number <= 1) {
```

```
    return number;
```

```
  }
```

```
  return number * calculate(number - 1);
```

```
};
```

```
const Factorial = (number) => {
```

```
  return calculate(number);
```

```
};
```

```
export default Factorial;
```


8. React. Hooks. Користувальницькі хуки.

Тепер будемо використовувати наш хук з іншої компоненти:

```
import React, { useState } from 'react';
import useFactorial from './useFactorial';

const App = () => {
  const [state, setState] = useState({ number: 0 });
  const factorial = useFactorial(state.number);
  return (
    <button
      onClick={() => setState(prevState => ({
        ...prevState,
        number: prevState.number + 1,
      })))}
    >
      {`!${state.number} = ${factorial}`}
    </button>
  );
};
```

9. React. Hooks. Користувальницькі хуки.

Код на попередньому слайді працює, однак у ньому є деяка проблема з продуктивністю. Наш хук викликається і рахує факторіал при кожному рендері...

Хоча у тому коді це не можна назвати проблемою, т.к. рендер спрацьовує тільки через зміну **state**, після натискання на кнопку інкременту змінної `number`. І якщо `number` змінився, то нам потрібно заново порахувати факторіал.

Але ось якби в тому прикладі було кілька факторів, які могли б викликати ре-рендер (наприклад ще одна кнопка, яка змінює інше поле нашого `state`), тоді факторіал би перераховувався при кожному новому рендері, а нам потрібно тільки тоді, коли змінюється значення змінної `number` (яку ми передаємо аргументом у **`useFactorial`**).

10. React. Hooks. useMemo.

Для вирішення проблеми оптимізації з минулого прикладу служить хук **useMemo**. Він, як і **useEffect**, приймає 2 аргументи:

- callback, у тілі якого буде код, який має бути виконаний, і результат виконання якого буде повернено із виклику useMemo;
- масив змінних, за значеннями яких “стежитиме” реакт. Якщо значення змінюються, то код із callback-функції буде знову виконано.

Оптимізований код використання нашого хука для обчислення факторіалу виглядатиме так:

```
import React, { useState, useMemo } from 'react';
import useFactorial from './useFactorial';
const App = () => {
  const [state, setState] = useState({ number: 0 });
  const factorial = useMemo(() => useFactorial(state.number), [state.number]);
  return (...);
};
```

11. React. Hooks. Redux.

Бібліотека **react-redux** також додала кілька зручних хуків до свого арсеналу.

Тепер для підключення редаксу до react-компонентів - їх не потрібно обгортати у функцію **connect**, щоб перекласти змінні зі стану **store**, в props компоненти, і додати props **dispatch**

Тепер усі ці завдання вирішують два хуки:

- **useSelector**
- **useDispatch**

12. React. Hooks. Redux. useSelector.

Приклад використання хука **useSelector**:

```
import React from 'react';
import { useSelector } from 'react-redux';

const App = () => {
  const {
    students,
    tickets,
  } = useSelector(state => state);
  return (...);
};
```

13. React. Hooks. Redux. useDispatch.

Приклад

використання

хука

useDispatch:

```
import React, { useEffect } from 'react';
import { useDispatch } from 'react-redux';
import studentsActions from '../actions/studentsActions';

const App = () => {
  const dispatch = useDispatch();

  useEffect(() => {
    dispatch(studentsActions.fetchStudents({
      studentsCount: 5,
    }));
  }, []);

  return (...);
};
```

14. React. Material-UI.

Material-ui — це набір готових компонентів інтерфейсу користувача (кнопки, поля введення, каруселі і т.д ...). Ознайомитися з цим продуктом можна за посиланням:

<https://material-ui.com/>

Встановлення цієї залежності у проект: **npm install @mui/material @emotion/react @emotion/styled**

15. React. Material-ui. Use their components.

Приклад використання компоненти з material-ui:

```
import React from "react";
import Button from '@mui/material/Button';

const App = () => {
  return (
    <div>
      <Button variant="contained" color="primary">
        Press me!
      </Button>
    </div>
  );
}

export default App;
```


16. React. JSS

JSS — більш потужна абстракція над CSS. Він використовує Javascript як мову для опису стилів у декларативному та зручному у підтримці вигляді. Це високопродуктивний компілятор з JS CSS, який працює і на стороні сервера, і на стороні клієнта в реальному часі. Це низькорівнева бібліотека, яка не прив'язана до будь-якого фреймворку.

Встановлення цієї залежності у проект: **`npm install react-jss`**

17. React. Use CSS-in-JS.

Приклад коду із використанням CSS-in-JS:

```
import React from "react";
import { createUseStyles } from 'react-jss';

const getClasses = createUseStyles(() => ({
  myRedText: {
    color: 'red',
  },
}));

const App = () => {
  const classes = getClasses();
  return (
    <div className={classes.myRedText}>
      My text with red color
    </div>
  );
}

export default App;
```